

Pick-and-Place with Local Planning Only

IRPP Course Project

Monishram Selvaraj

2024102076

Praveen Vijay

2024102019

Gaurav Reddy

2024112006

1 Introduction and Problem Statement

This project involves implementation a reactive pick and place concept using a robotic arm (Universal Robot-5) with a local planning technique (APF). We use a depth camera mounted on the wrist of the arm (Kinect camera), to produce the required pointcloud.

1.1 Project Objectives

1. Pick an object, known target object pose
2. Transports to a target location
3. Does not utilize precomputed joint trajectories
4. Performs reactive obstacle avoidance with an APF (Artificial Potential Field) concept

The simulation videos for this project can be accessed [here](#).

2 Setup and Softwares Used

2.1 Simulation Setup

The pick and place task is performed in a simulated tabletop environment. The arm is placed on the table surface, which also contains a target object to be picked up and additional obstacles placed throughout the table to demonstrate the reactive avoidance behavior.

The object pose is assumed to be known and provided directly to the manipulation pipeline. During execution, the robot performs real time motion planning while avoiding collisions with the obstacles.

2.2 Softwares Used

- **ROS 2 Humble** – Middleware for communication between robotic modules.
- **Gazebo** – Physics-based simulation environment.
- **RViz2** – Visualization of robot state and planning scene.
- **MoveIt 2** – Provides kinematics and collision checking.
- **MoveIt Servo** – Enables real-time reactive control.
- **URDF** – Robotic arm description format.

3 Methodology

3.1 Robotic Arm Model

The manipulation task is performed using a Kinova Jaco Kinect robotic arm, a 6-DOF arm. The robot operates in joint space but performs task execution in Cartesian space through end-effector control. The robot is modeled using a URDF description that defines the kinematics chain, joint limits, and collision geometry required for the simulation and planning of the problem.

3.2 Obstacle Detection

A Kinect depth camera is mounted on the manipulator which captures the surrounding environment and generates a point cloud for the workspace, which contains information about the target object as well as the nearby obstacles. Portions of the robot itself may also appear in the captured point cloud, and the system might incorrectly interpret parts of the manipulator as obstacles. To prevent this, the robot's URDF (Unified Robot Description Format) model, which contains the geometric description of the robot links, is used to remove points corresponding to the robot body. The resulting filtered point cloud provides an accurate representation of the external obstacles in the environment.

3.3 Local Planning

Since we're not meant to rely on precomputed global trajectories, we employed a local planning strategy based on the Artificial Potential Field (APF) method. In this approach, the robot's surrounding is modeled as a potential field in which the target generates an attractive potential while obstacles generate repulsive potentials. A net force is then calculated, using which we move the end-effector.

The attractive potential pulls the end-effector toward the desired goal position. This force is defined as

$$F_{att} = k_{att}(x_{goal} - x)$$

where x represents the current end-effector position, x_{goal} represents the target position, and k_{att} is a gain constant controlling the strength of the attraction.

Obstacles produce a repulsive force that pushes the manipulator away from potential collisions. This force is activated only when the robot is within a certain distance d_0 from an obstacle and is defined as

$$F_{rep} = \begin{cases} k_{rep} \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2} \nabla d, & d < d_0 \\ 0, & d \geq d_0 \end{cases}$$

where d represents the distance between the robot and the obstacle.

The total force acting on the end-effector is obtained by combining the attractive and repulsive components:

$$F_{total} = F_{att} + \sum F_{rep}$$

This formulation allows the robot to continuously update its motion in response to nearby obstacles, enabling smooth and reactive navigation in dynamic environments without requiring global trajectory replanning.

3.4 Kinematic Control

The force vector produced by the APF planner is interpreted as a desired end-effector velocity in Cartesian space. Since the robot is controlled through joint commands, this Cartesian velocity must be converted into joint-space velocities before being sent to the robot controller.

This conversion is performed using Jacobian-based velocity control:

$$\dot{q} = J^{-1}(q)\dot{x}$$

where $\dot{x}$ represents the calculated end-effector velocity and $\dot{q}$ represents the corresponding joint velocity commands.

3.5 System Pipeline

The perception system operates at a refresh rate of approximately 30 Hz, corresponding to the update frequency of the depth camera (30 frames per second). In contrast, the control loop and local planner operate at a higher frequency of 100 Hz in order to ensure smooth and responsive motion.

Between sensor updates, the planner continues to use the most recently available obstacle information to compute motion commands. This decoupled sensing and control frequency enables the robot to maintain stable motion while still responding to dynamic changes in the environment.

The resulting system pipeline can be summarized as:

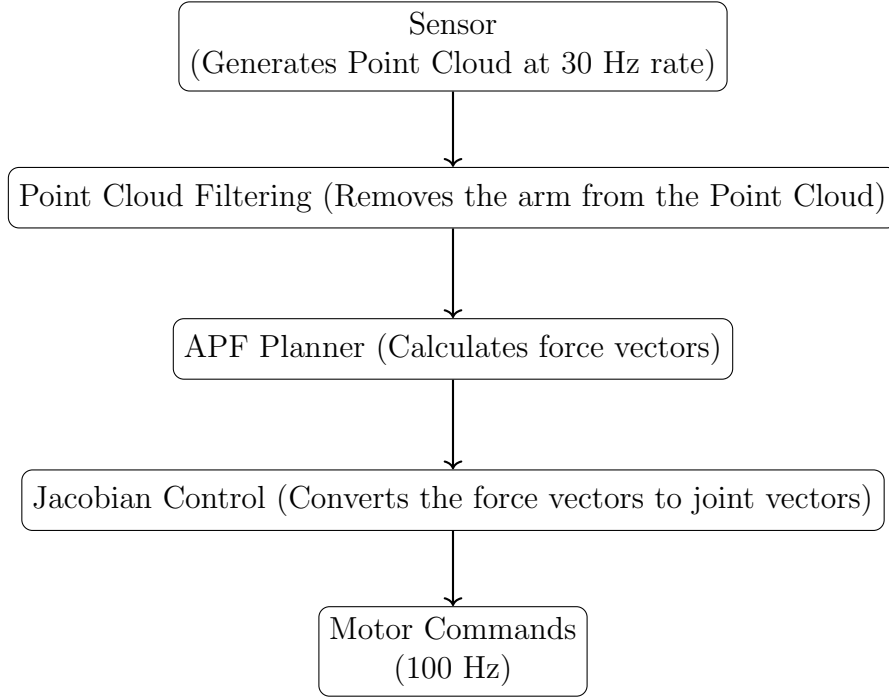


Figure 1: System pipeline

4 Implementation

As mentioned, we’ve implemented the system in Gazebo and RViz2. This section describes the implementation, and includes pictures for the individual stages/components.

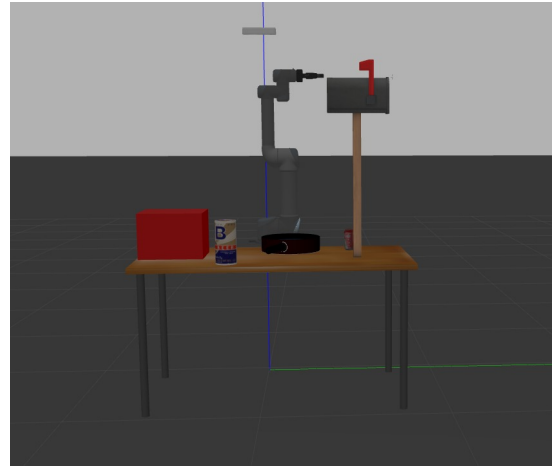
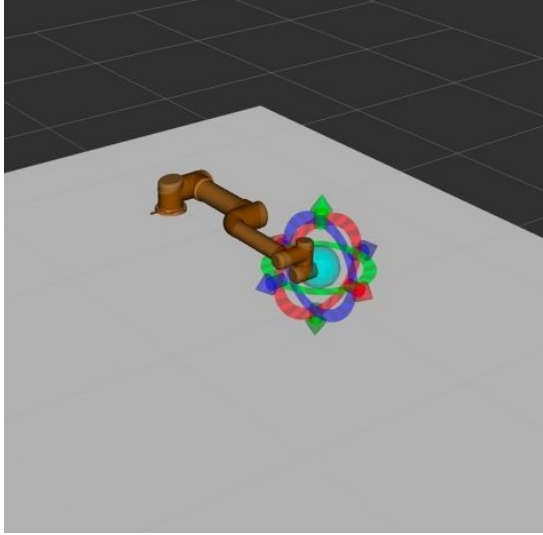


Figure 2: UR5 robotic arm and basic tabletop setup

The arm is placed atop a table, and the Kinect depth camera is hovering on top, which has a full view from above, collecting all the data for the pointcloud generation. As discussed earlier, this will also include the robot's arm, so filtering that out needs to be taken care of.

4.1 Implementation Challenge

During implementation, one issue that arose was related to the arm actually grabbing the target object. Since the objects on Gazebo are defaulted to having zero friction, simply closing the gripper will fling the object away. To solve this, we included another script that just directly attaches the robot's wrist link to the object link (links are properties of the objects in Gazebo).

5 Running it Locally

A zip file is provided as a submission, which has the following structure:

```
|-- deps.repos
|-- docker
|   |-- Dockerfile
|   '-- entrypoint.sh
'-- src
    |-- IFRA_LinkAttacher
    |   |-- LICENSE.md
    |   |-- linkattacher_msgs
    |   |-- README.md
    '-- ros2_LinkAttacher
```

```

|-- ur5_camera_gripper_moveit_config
|   |-- CMakeLists.txt
|   |-- config
|   |-- launch
|   '-- package.xml
'-- ur5_simulation
    |-- CMakeLists.txt
    |-- config
    |-- launch
    |-- meshes
    |-- models
    |-- package.xml
    |-- rviz
    |-- scripts
    |-- urdf
    '-- worlds

```

5.1 Source the ROS Environment

First, source the ROS2 environment and the workspace.

```

source /opt/ros/humble/setup.bash
source install/setup.bash

```

If using the docker container, source the container environment as well.

5.2 Launch the Simulation

Launch the Gazebo world along with the MoveIt and RViz configuration.

```

gazebo /workspaces/app_ws/src/ur5_simulation/worlds/world2.world &
ros2 launch ur5_camera_gripper_moveit_config demo.launch.py

```

This launches the Gazebo simulation environment and opens RViz for visualization.

5.3 Enable Simulation Time

Set the required parameters so that all components use simulation time.

```

ros2 param set /joint_trajectory_controller use_sim_time true
ros2 param set /controller_manager use_sim_time true
ros2 param set /move_group use_sim_time true

```

5.4 Test Arm Movement

The following command can be used to test whether the arm is responding to trajectory commands.

```
ros2 action send_goal /joint_trajectory_controller/follow_joint_trajectory \
control_msgs/action/FollowJointTrajectory \
"{trajectory:
{header: {frame_id: 'world'},
joint_names: ['shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint',
'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint'],
points: [{positions: [0.5, -1.57, 0.0, -1.57, 0.0, 0.0]},
velocities: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]},
time_from_start: {sec: 3}}}]}"
```

This command sends a simple joint trajectory to verify that the robot arm is functioning correctly.

5.5 Attach Object to Gripper

To simulate grasping the object, the following service call can be used:

```
ros2 service call /ATTACHLINK linkattacher_msgs/srv/AttachLink \
"{model1_name: 'model1', link1_name: 'link1',
model2_name: 'model2', link2_name: 'link2'}"
```

5.6 Detach Object from Gripper

To release the object after placement:

```
ros2 service call /DETACHLINK linkattacher_msgs/srv/DetachLink \
"{model1_name: 'model1', link1_name: 'link1',
model2_name: 'model2', link2_name: 'link2'}"
```

If pick-and-place execution is required, these commands should be executed in a new terminal where the ROS workspace has already been sourced.